

Lisp, 3rd Edition

Patrick Henry Winston and Berthold Klaus Paul Horn

Chapter 1 Understanding Symbol Manipulation

- Symbol Manipulation Is Like Working with Words and Sentences
- LISP Helps Make Computers Intelligent
- LISP Promotes Productivity and Facilitates Education
- LISP Is the Right Symbol-Manipulation Language To Learn
- COMMON LISP Is the Right LISP To Learn
- Beware of False Myths
- Summary
- References

Chapter 2 Basic LISP Primitives

- LISP Means Symbol Manipulation
- LISP Procedures and Data Are Symbolic Expressions
- Problems
- Lists Are Like Bowls
- FIRST and REST Take Lists Apart
- Quoting Stops Evaluation
- Problems
- Some Old Timers Use CARs and CDRs
- SETF Assigns Values to Symbols
- SETF Accepts Multiple Symbol-Value Pairs
- Certain Atoms Evaluate to Themselves
- CONS, APPEND, and LIST Construct Lists
- Problems
- CONS, APPEND, and LIST Do Not Alter Symbol Values
- NTHCDR, BUTLAST, and LAST Shorten Lists
- LENGTH and REVERSE Work on Top-Level Elements
- Problems
- ASSOC Looks for Indexed Sublists

- LISP Offers Integers, Ratios, and Floating-Point Numbers, among Others
- A Few Primitives for Numbers Round Out a Basic Repertoire
- Problems
- Summary

Chapter 3 Procedure Definition and Binding

- DEFUN is LISP's Procedure-Definition Primitive
- Parameter Variable Values Are Isolated by Virtual Fences
- Special Variable Values Are Not Isolated by Virtual Fences
- Procedures Match Parameters to Arguments
- Problems
- LET Forms Bind Parameters to Initial Values
- LET Forms Produce Nested Fences
- LET Forms Evaluate Initial-Value Forms in Parallel
- LET* Forms Evaluate Initial-Value Forms Sequentially
- Progressive Envelopment and Comment Translation Help Define New Procedures
- Summary

Chapter 4 Predicates and Conditionals

- A Predicate Is a Procedure That Returns True or False
- EQUAL, EQ, EQL, and = Are Equality Predicates
- MEMBER Tests for List Membership
- Keyword Arguments Modify Behavior
- LISTP, ATOM, NUMBERP, and SYMBOLP Are Data-Type Predicates
- NIL Is Equivalent to the Empty List
- NULL and ENDP Are Empty-List Predicates
- There Are Many Number Predicates
- Problems
- AND, OR, and NOT Enable Elaborate Testing
- Problems
- Predicates Help IF, WHEN, and UNLESS Choose among Alternatives
- Problems

- Predicates Help COND Choose among Alternatives
- Problems
- CASE Is Still Another Conditional
- Conditionals Enable DEFUN To Do Much More
- Problems
- Problem Reduction Helps Define New Procedures
- Summary

Chapter 5 Procedure Abstraction and Recursion

- Procedure Abstraction Hides Details Behind Abstraction Boundaries
- Recursion Allows Procedures To Use Themselves
- Recursion Can Be Efficient
- Problems
- Recursion Can Be Used To Analyze Nested Expressions
- Problems
- Optional Parameters Eliminate the Need for Many Auxiliaries
- Problems
- Advanced Programmers Use Rest, Key, and Aux Parameters
- Only a Few LISP Primitives Are Really Necessary
- Problems
- Summary
- References

Chapter 6 Data Abstraction and Mapping

- Data Details Stifle Progress
- Data Abstraction Facilitates Progress
- You Should Use Readers, Constructors, and Writers Liberally
- It Is Useful To Transform and To Filter
- Recursive Procedures Can Transform and Filter
- Recursive Procedures Can Count and Find
- Problems
- Clichés Embody Important Programming Knowledge
- MAPCAR Simplifies Transforming Operations

- REMOVE-IF and REMOVE-IF-NOT Simplify Filtering Operations
- COUNT-IF and FIND-IF Simplify Counting and Finding Operations
- FUNCALL and APPLY Also Take a Procedure Argument
- LAMBDA Defines Anonymous Procedures
- Summary
- References

Chapter 7 Iteration on Numbers and Lists

- DOTIMES Supports Iteration on Numbers
- Problems
- DOLIST Supports Iteration on Lists
- Problems
- DO Is More General than DOLIST and DOTIMES
- Problems
- LOOP Never Stops, Almost
- PROG1 and PROGN Handle Sequences Explicitly
- Problems and Their Representation Determine Proper Control Strategy
- Summary

Chapter 8 File Editing, Compiling, and Loading

- Programs and Data Reside in Files
- File Specifications Tend to Have Baroque Forms
- ED Takes You to an Editor
- EMACS Is a Particularly Powerful LISP Editor
- COMPILE-FILE Compiles Files
- LOAD Causes LISP To Read from Files
- Summary
- References

Chapter 9 Printing and Reading

- PRINT and READ Facilitate Simple Printing and Reading
- FORMAT Enables Exotic Printing
- Problems

- `WITH-OPEN-FILE` Enables Reading from Files
- Optional Arguments in `READ` Forms Specify End-of-File Treatment
- `WITH-OPEN-FILE` Enables Printing to Files
- Problems
- `READ` Does Not Evaluate Expressions, but `EVAL` Evaluates Twice
- Problems
- Special Primitives Manipulate Strings and Characters
- `READ-LINE` and `READ-CHAR` Read Strings and Characters
- Summary

Chapter 10 Rules for Good Programming and Tools for Debugging

- Following Rules of Good Programming Practice Helps You To Avoid Bugs
- Big Programs Require Abstraction and Modularity
- Most Programmers use `TRACE`, `STEP`, and `BREAK` with Varying Frequency
- `TRACE` Causes Procedures To Print Their Arguments and Values
- `STEP` Causes Procedures To Proceed One Step at a Time
- `BREAK` Stops Evaluation so that You Can Evaluate Forms
- `TIME`, `DESCRIBE`, and `DRIBBLE` Are Helpful Too
- Debugging Is Implementation Specific
- Summary
- References

Chapter 11 Properties and Arrays

- Each Way of Storing Data Has Constructors, Readers, and Writers
- Properties Enable Storage in Symbolically Indexed Places
- `GET` and `SETF` are the Custodians of Properties
- Problems
- Arrays Enable Storage in Numerically Indexed Places
- `MAKE-ARRAY`, `AREF`, and `SETF` are the Custodians of Arrays
- Problems
- Summary

Chapter 12 Macros and Backquote

- Macros Translate and Then Evaluate
- The Backquote Mechanism Simplifies Template Filling
- The Backquote Mechanism Simplifies Macro Writing
- Optional, Rest, and Key Parameters Enable More Powerful Macros
- Problems
- Macros Deserve Their Own File
- Summary

Chapter 13 Structures

- Structure Types Facilitate Data Abstraction
- Structure Types Enable Storage in Procedurally Indexed Places
- Individual Structure Types Are New Data Types
- Problems
- One Structure Type Can Include the Fields of Another
- Problems
- Structure Types Are Important Components of Big Systems
- DESCRIBE Prints Descriptions
- DEFSTRUCTs Deserve Their Own File
- Summary

Chapter 14 Classes and Generic Functions

- What to Do Depends on What You Do it to
- You Can Make Ordinary Procedures Data Driven, Albeit Awkwardly
- Methods Are Procedures Selected from Generic Functions by Argument Types
- Classes Resemble Structure Types but Resonate Better with Generic Functions
- Any Nonoptional Argument's Class Can Help Select a Method
- Classes Enable Method Inheritance
- Problems
- The Most Specific Method Takes Precedence over the Others
- Parameter Order Helps Determine Method Precedence
- Simple Rules Approximate the Complicated Class Precedence Algorithm
- Problems
- Methods Can Be Specialized to Individual Instances

- Problems
- Method Selection Involves Three Steps
- Object-Oriented Programming Offers Advantages, Not Magic
- Summary
- References

Chapter 15 Lexical Variables, Generators, and Encapsulation

- LETs Produce Nested Fences
- Nested Fences Provide Variable Values
- Procedure Calls Usually Do Not Produce Nested Fences
- Problems
- Nested Definitions do Produce Nested Fences
- Generators Produce Sequences
- Nameless Procedures Produce Nested Fences
- Nameless Procedures Can Be Assigned to Variables
- The Free Variables in Nameless Procedures Can Be Encapsulated
- Encapsulation Enables the Creation of Sophisticated Generators
- Generators Can Be Defined by other Procedures
- Nameless Procedures Become Lexical Closures
- Summary
- References

Chapter 16 Special Variables

- Bindings Could Be Kept on a Record of Calls
- Some Variables Are Declared To Be Special Forevermore
- Special-Variable Bindings Are Actually Kept on a Stack
- DEFVAR Can Assign as Well as Declare
- Some Variable Instances Can Be Special while Others Are Lexical
- Both Lexical and Special Variables Can Be Free Variables
- Summary

Chapter 17 List Storage, Surgery, and Reclamation

- Lists Can Be Represented by Boxes and Pointers

- Boxes and Pointers Can Be Represented by Bytes
- CONS Builds New Lists by Depositing Pointers in Free Boxes
- APPEND Builds New Lists by Copying
- NCONC and DELETE Can Alter Box Contents Dangerously
- SETF Also Can Alter Box Contents Dangerously
- Problems
- EQ Checks Pointers Only
- Garbage Collection Reclaims Abandoned Memory
- LISP Allows You To Write Inefficient Procedures
- Problems
- Simple Garbage Collectors Use the Mark and Sweep Approach
- Simulation Procedures Expose Garbage Collection Details
- MARK Places Marks on Useful Chunks
- SWEEP Collects Unmarked Chunks
- Marking Can Be Done without Recursion
- Our Nonrecursive Marking Procedure Leaves a Trail of Pointers
- Some Garbage Collectors Are Incremental
- Summary

Chapter 18 LISP in LISP

- It Is Easy To Build a Simple Interpreter for a LISPlike Language
- MICRO-EVAL and MICRO-APPLY Work Together To Evaluate Forms
- Traces Show How MICRO-EVAL and MICRO-APPLY Work Together
- Problems
- Closures Encapsulate Environments
- Problems
- Special Variable Binding Can Be Arranged
- LISP Does Call-by-Value Rather Than Call-by-Reference
- LISP Can Be Defined in LISP
- Fancy Control Structures Usually Start Out as Basic LISP Interpreters
- Summary

Chapter 19 Examples Involving Search

- Breadth-First and Depth-First Searches Are Basic Strategies
- Problems
- Best-First Search and Hill-Climbing Require Sorting
- Problems
- Problems about Measuring Out a Volume of Water
- Problems about Placing Queens on a Chess Board
- Summary
- References

Chapter 20 Examples Involving Simulation

- Projects Involve Events and Tasks
- Structures Can Represent Events and Tasks
- Simulation Procedures Can Propagate Event Times
- Event and Task Structures Require Special Printing Procedures
- An Event List Keeps Simulation in Step with the Simulated Project
- Problems
- Summary

Chapter 21 The Blocks World with Classes and Methods

- The Blocks-World Program Handles Put-On Commands
- Object-Oriented Programming Shifts Attention from Procedures to Objects
- Object-Oriented Programming Begins with Class Specification
- Problems
- Slot Readers Are Generic Functions
- The Blocks-World Program's Methods Are Transparent
- Before and After Methods Simplify Bookkeeping
- Problems
- Slot Writers Are Generic Functions
- Object-Oriented Programming Enables Automatic Procedure Assembly
- You Can Control How Instances Are Printed
- The Number-Crunching Methods Can Be Ignored
- Problems
- The Blocks-World Program Illustrates Abstraction

- Summary
- References

Chapter 22 Answering Questions about Goals

- The Blocks-World Program Can Introspect into its Own Operation
- Remembering Generic Function Calls Creates a Goal Tree
- Macros Enable Method-Defining Procedures To Be Defined
- Problems
- The Goal Tree Is Easy to Display
- The Goal Tree Answers Questions
- Problems
- Summary
- References

Chapter 23 Constraint Propagation

- Constraints Propagate Numbers through Arithmetic Boxes
- Constraints Propagate Probability Bounds through Logic Boxes
- Classes Represent Assertions and Logical Constraints
- Generic Functions Enforce Constraints
- More Information Moves Probability Bounds Closer
- Problems
- Summary
- References

Chapter 24 Symbolic Pattern Matching

- Matching Compares Patterns and Datums Element by Element
- **MATCH** Keeps Variable Bindings on an Association List
- Matching Is Easily Implemented by a Recursive Procedure
- Matching Is Better Implemented Using Procedure Abstraction
- Problems
- Unification Is Generalized Matching
- Summary
- References

Chapter 25 Streams and Delayed Evaluation

- Streams Are Sequences of Data Objects
- We Can Represent Streams Using Lists
- We Can Delay Evaluation by Encapsulation
- We Can Represent Streams Using Delayed Evaluation
- Problems
- Summary
- References

Chapter 26 Rule-Based Expert Systems and Forward Chaining

- Forward Chaining Means Working from Antecedents to Consequents
- We Use Streams To Represent Assertions and Rules
- Our First Pass Concentrates on **MATCH** and the Binding Stream
- Our Second Pass Concentrates on the Procedures that Surround **MATCH**
- Simple Rules Help Identify Animals
- Rules Facilitate Question Answering and Probability Computing
- Our Forward-Chaining Program Illustrates Abstraction
- Summary
- References

Chapter 27 Backward Chaining and **PROLOG**

- Our Backward Chainer Borrows Procedures from our Forward Chainer
- Backward Chaining Means Working from Consequents to Antecedents
- Our First Pass Concentrates on **MATCH**, **UNIFY**, and the Binding Stream
- Our Second Pass Concentrates on the Procedures that Surround **MATCH** and **UNIFY**
- Completing Our Backward-Chaining Program Involves a Few Auxiliary Procedures
- Simple Rules Help Identify Animals
- Problems
- Our Backward Chainer Implements a Language like **PROLOG**
- Problems
- Our Backward-Chaining Program Illustrates Abstraction

- Summary
- References

Chapter 28 Interpreting Transition Trees

- Procedures Can Produce Multiple Values
- Natural-Language Interfaces Produce Database Commands
- Transition Trees Capture English Syntax
- A Transition Tree Interpreter Follows an Explicit Description
- Multiple-Valued Procedures Embody Transition Trees
- Our Interpreter Uses Explicit Transition-Tree Descriptions
- We Use a Macro To Simplify Tree Definition
- A Read, Analyze, and Report Loop Adds a Finishing Touch
- Problems
- Summary
- References

Chapter 29 Compiling Transition Trees

- Transition Trees Can Be Compiled from Transparent Specifications
- Compilers Treat Programs as Data
- Compiled Programs Run Faster
- Compilers Are Usually Major Undertakings
- LISP Itself Is Either Compiled or Interpreted
- Problems
- Summary

Chapter 30 Procedure-Writing Programs and Database Interfaces

- Grammars Can Be Sophisticated
- Answering Requests Is Done in Three Steps
- Most Database Commands Transform Relations into Relations
- English Questions Correspond to Database Commands
- Our Simulated Database Supports an Improved Grammar
- Problems
- The Relational Database Can Be Faked

- Problems
- The Database Illustrates Data Abstraction
- Summary

Chapter 31 Finding Patterns in Images

- Generating All Possible Matches Helps Isolate the Correct Match
- Constraints Are Needed To Isolate the Correct Match
- The Search Tree Can Be Pruned Using Geometric Information
- Matches Have To Be Checked for Global Consistency
- Matching Is Harder if Mismatches Are Allowed
- Keeping Track of Mismatches Improves Efficiency
- The Cost of Filtering Has To Be Weighed against the Cost of Searching
- Multiple Matching Attempts Lead Recognition
- Problems
- Edges Provide More Constraint than Points
- Problems
- Summary
- References

Chapter 32 Converting Notations, Manipulating Matrices, and Finding Roots

- It Is Easy to Translate Infix Notation to Prefix
- Problems
- Sparse Matrices Can Be Represented as Lists of Lists
- Problems
- Complex Numbers Constitute Another Numeric Data Type
- Roots of Quadratic Equations Are Easy To Calculate
- Roots of Cubic Equations Can Be Calculated
- Roots of Quartic Equations Are Harder To Calculate
- Problems
- Summary
- References

Appendix: The Computation of the Class Precedence List

- Make Initial Lists
- Make a List of Precedence Pairs
- Make a List of Precedence List Candidates
- Select a Candidate
- Shrink the List of Precedence Pairs
- Repeat

Solutions

Glossary

Bibliography

Index of LISP Primitives Used in this Book

Index of LISP Definitions

General Index